

6. Use the following small text dataset to train a simple Variational Autoencoder (VAE) for text reconstruction:

["The sky is blue", "The sun is bright", "The grass is green", "The night is dark", "The stars are shining"]

1. Preprocess the data (tokenize and pad the sequences).
2. Build a basic VAE model for text reconstruction.
3. Train the model and show how it reconstructs or generates similar sentences.

```
1 !pip install tensorflow numpy
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!>=0.5.1,!>=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.1)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.2.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.81.1)
Requirement already satisfied: tensorboard~>2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow) (0.42.0)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (2025.11.11)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~>2.20.0->tensorflow) (3.7.0)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorboard~>2.20.0->tensorflow) (11.0.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboard~>2.20.0->tensorflow) (0.19.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~>2.20.0->tensorflow) (3.1.0)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboa) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow) (2.19.0)
Requirement already satisfied: mdurl~>0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras) (0.1.2)
```

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5
6 # 1. PREPROCESSING THE DATA
7 texts = ["The sky is blue", "The sun is bright", "The grass is green",
8          "The night is dark", "The stars are shining"]
9
10 # Lowercase and tokenize by splitting on spaces
11 tokens = [t.lower().split() for t in texts]
12
13 # Build Vocabulary with special tokens
14 vocab = {"<pad>": 0, "<sos>": 1, "<eos>": 2}
15 for seq in tokens:
16     for word in seq:
17         if word not in vocab:
18             vocab[word] = len(vocab)
19
20 idx2word = {i: w for w, i in vocab.items()}
21 vocab_size = len(vocab)
22
23 # Pad sequences and add Start-Of-Sequence (<sos>) and End-Of-Sequence (<eos>)
24 max_len = max(len(seq) for seq in tokens)
25 data = []
26 for seq in tokens:
27     encoded = [vocab["<sos>"]] + [vocab[w] for w in seq] + [vocab["<eos>"]]
28     # Add padding to make all sentences the same length
29     encoded += [vocab["<pad>"]] * (max_len - len(seq))
30     data.append(encoded)
31
32 # Convert to PyTorch tensor
33 data_tensor = torch.tensor(data) # Shape: (5, max_len + 2)
```

```

34
35
36 # 2. BUILDING THE VAE MODEL
37 class TextVAE(nn.Module):
38     def __init__(self, vocab_size, embed_dim=16, hidden_dim=32, latent_dim=8):
39         super().__init__()
40         self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
41
42         # Encoder: reads the sequence and outputs a hidden state
43         self.encoder_gru = nn.GRU(embed_dim, hidden_dim, batch_first=True)
44
45         # Latent Space: maps hidden state to Mean (mu) and Variance (log_var)
46         self.fc_mu = nn.Linear(hidden_dim, latent_dim)
47         self.fc_var = nn.Linear(hidden_dim, latent_dim)
48
49         # Decoder: maps latent space back to hidden state to generate text
50         self.fc_z = nn.Linear(latent_dim, hidden_dim)
51         self.decoder_gru = nn.GRU(embed_dim, hidden_dim, batch_first=True)
52         self.fc_out = nn.Linear(hidden_dim, vocab_size)
53
54     def encode(self, x):
55         embedded = self.embed(x)
56         _, hidden = self.encoder_gru(embedded)
57         hidden = hidden.squeeze(0) # Remove RNN layer dimension
58         mu = self.fc_mu(hidden)
59         log_var = self.fc_var(hidden)
60         return mu, log_var
61
62     def reparameterize(self, mu, log_var):
63         # Trick to allow backpropagation through random sampling
64         std = torch.exp(0.5 * log_var)
65         eps = torch.randn_like(std)
66         return mu + eps * std
67
68     def forward(self, x):
69         # 1. Encode
70         mu, log_var = self.encode(x)
71         # 2. Sample from latent space
72         z = self.reparameterize(mu, log_var)
73
74         # 3. Decode (using Teacher Forcing for training)
75         hidden = self.fc_z(z).unsqueeze(0)
76         dec_input = x[:, :-1] # Input everything except the last token
77         embedded = self.embed(dec_input)
78         out, _ = self.decoder_gru(embedded, hidden)
79         logits = self.fc_out(out)
80
81         return logits, mu, log_var
82
83     def generate(self, z, max_len):
84         # Generate text iteratively from a latent vector (z)
85         hidden = self.fc_z(z).unsqueeze(0)
86         curr_token = torch.tensor([[vocab["<sos>"]]])
87         generated_words = []
88
89         for _ in range(max_len):
90             embedded = self.embed(curr_token)
91             out, hidden = self.decoder_gru(embedded, hidden)
92             logits = self.fc_out(out)
93
94             # Get the most likely next word
95             next_token = logits.argmax(-1)
96             curr_token = next_token
97
98             word_idx = next_token.item()
99             if word_idx == vocab["<eos>"]: break
100             generated_words.append(idx2word[word_idx])
101
102         return " ".join(generated_words)
103
104
105 # 3. TRAINING AND RECONSTRUCTION
106 model = TextVAE(vocab_size)
107 optimizer = optim.Adam(model.parameters(), lr=0.01)
108
109 print("Starting Training...")
110 for epoch in range(1, 201):
111     optimizer.zero_grad()
112     logits, mu, log_var = model(data_tensor)
113
114     # Target is shifted by 1 (we predict the next word)
115     targets = data_tensor[:, 1:]

```

```

116
117     # Reconstruction Loss (Cross Entropy)
118     recon_loss = F.cross_entropy(logits.reshape(-1, vocab_size), targets.reshape
119                                   (-1), ignore_index=0)
120
121     # KL Divergence Loss (forces latent space to be a normal distribution)
122     kl_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp()) /
123               data_tensor.size(0)
124
125     # Total Loss (using a small weight for KL to prevent posterior collapse)
126     loss = recon_loss + 0.1 * kl_loss
127
128     loss.backward()
129     optimizer.step()
130
131     if epoch % 50 == 0:
132         print(f"Epoch {epoch}/200 | Loss: {loss.item():.4f}")
133
134     print("\n--- Reconstructing Original Sentences ---")
135     with torch.no_grad():
136         for i in range(len(data_tensor)):
137             x = data_tensor[i:i+1]
138             mu, _ = model.encode(x)
139             # Using mu directly for deterministic reconstruction
140             reconstructed = model.generate(mu, max_len=max_len+2)
141             print(f"Original:      {texts[i]}")
142             print(f"Reconstructed: {reconstructed}\n")

```

Starting Training...

Epoch 50/200 | Loss: 0.3295

Epoch 100/200 | Loss: 0.2789

Epoch 150/200 | Loss: 0.2608

Epoch 200/200 | Loss: 0.2734

--- Reconstructing Original Sentences ---

Original: The sky is blue

Reconstructed: the sky is blue

Original: The sun is bright

Reconstructed: the sun is bright

Original: The grass is green

Reconstructed: the grass is green

Original: The night is dark

Reconstructed: the night is dark

Original: The stars are shining

Reconstructed: the stars are shining

7. Use a pre-trained GPT model (like GPT-2 or GPT-3) to translate a short English paragraph into French and German. Provide the original and translated text.

```
1 !pip install transformers torch
```

```

Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.12.1)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.11.0+cu128)
Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (1.5.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.23.0)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from transformers) (0.25.1)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.8.0)
Requirement already satisfied: tqdm>=4.60 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.29.4)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cuda-bindings] in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch) (9.19.0.56)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cuda-bindings] in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.7)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)

```

```
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit)
Requirement already satisfied: nvidia-cusparsesolver-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit)
Requirement already satisfied: nvidia-cuda-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9)
Requirement already satisfied: click>=8.4.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.13.3)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (1.5.4)
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (0.0.2)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: idna in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer->transformers)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=13.8.0->typer->transformers)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->typer->transformers)
```

```
1 from transformers import pipeline, set_seed
2
3 # Initialize the text-generation pipeline with the local GPT-2 model
4 # This will download the open-source model to your machine the first time you run it
5 generator = pipeline('text-generation', model='gpt2')
6 set_seed(42)
7
8 original_text = (
9     "Artificial intelligence is transforming the way we interact with technology. "
10    "From natural language processing to computer vision, these advancements are "
11    "making our daily tasks easier and more efficient."
12)
13
14 # We use "Few-Shot Prompting" to teach GPT-2 how to translate on the fly
15 french_prompt = (
16    "Translate English to French.\n"
17    "English: Hello, how are you?\n"
18    "French: Bonjour, comment allez-vous?\n"
19    "English: " + original_text + "\n"
20    "French:"
21)
22
23 german_prompt = (
24    "Translate English to German.\n"
25    "English: Hello, how are you?\n"
26    "German: Hallo, wie geht es dir?\n"
27    "English: " + original_text + "\n"
28    "German:"
29)
30
31 # Generate the translations
32 print("Generating French translation...")
33 french_output = generator(french_prompt, max_new_tokens=50, num_return_sequences=1, pad_token_id=50256)
34
35 print("Generating German translation...")
36 german_output = generator(german_prompt, max_new_tokens=50, num_return_sequences=1, pad_token_id=50256)
37
38 # Extract only the newly generated text
39 french_text = french_output[0]['generated_text'].split("French:")[1].strip()
40 german_text = german_output[0]['generated_text'].split("German:")[1].strip()
41
42 print("\n--- Original English Text ---")
43 print(original_text)
44 print("\n--- French Translation ---")
45 print(french_text)
46 print("\n--- German Translation ---")
47 print(german_text)
```

Loading weights: 100% 148/148 [00:00<00:00, 2635.54it/s]

[transformers] Both `max\_new\_tokens` (=50) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence.
Generating French translation...

[transformers] Both `max\_new\_tokens` (=50) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence.
Generating German translation...

--- Original English Text ---
Artificial intelligence is transforming the way we interact with technology. From natural language processing to computer vision, these advancements are making our daily tasks easier and more efficient.

--- French Translation ---
I'm

--- German Translation ---
I would like to know your name.

```

1 from transformers import pipeline
2
3 # We use the standard 'text-generation' pipeline with a modern, small Causal LM
4 # Qwen2.5-0.5B-Instruct is a tiny open-source model that excels at instructions
5 translator = pipeline('text-generation', model='Qwen/Qwen2.5-0.5B-Instruct')
6
7 original_text = (
8     "Artificial intelligence is transforming the way we interact with technology. "
9     "From natural language processing to computer vision, these advancements are "
10    "making our daily tasks easier and more efficient."
11 )
12
13 messages_fr = [
14     {"role": "system", "content": "You are a professional translator."},
15     {"role": "user", "content": f"Translate this text into French:\n\n{original_text}"}
16 ]
17
18 print("Generating French translation...")
19 french_output = translator(messages_fr, max_new_tokens=100)
20
21 messages_de = [
22     {"role": "system", "content": "You are a professional translator."},
23     {"role": "user", "content": f"Translate this text into German:\n\n{original_text}"}
24 ]
25
26 print("Generating German translation...")
27 german_output = translator(messages_de, max_new_tokens=100)
28
29
30 print("\n--- Original English Text ---")
31 print(original_text)
32
33 print("\n--- French Translation ---")
34 print(french_output[0]['generated_text'][-1]['content'].strip())
35
36 print("\n--- German Translation ---")
37 print(german_output[0]['generated_text'][-1]['content'].strip())

```

```

config.json: 100%                                659/659 [00:00<00:00, 70.3kB/s]

model.safetensors: 100%                          988M/988M [00:12<00:00, 92.2MB/s]

Loading weights: 100%                            290/290 [00:00<00:00, 4056.02it/s]

generation_config.json: 100%                      242/242 [00:00<00:00, 22.2kB/s]

tokenizer_config.json: 100%                       7.30k/7.30k [00:00<00:00, 608kB/s]

vocab.json: 100%                                  2.78M/2.78M [00:00<00:00, 58.2MB/s]

merges.txt: 100%                                  1.67M/1.67M [00:00<00:00, 35.1MB/s]

tokenizer.json: 100%                              7.03M/7.03M [00:00<00:00, 96.0MB/s]

[transformers] Both `max_new_tokens` (=100) and `max_length` (=20) seem to have been set. `max_new_tokens` will take precedence.
Generating French translation...
[transformers] Ignoring clean_up_tokenization_spaces=True for BPE tokenizer Qwen2Tokenizer. The clean_up_tokenization post-processor will be ignored.
[transformers] Both `max_new_tokens` (=100) and `max_length` (=20) seem to have been set. `max_new_tokens` will take precedence.
Generating German translation...

--- Original English Text ---
Artificial intelligence is transforming the way we interact with technology. From natural language processing to computer vision, AI is revolutionizing the way we live and work.

--- French Translation ---
L'intelligence artificielle est transformant notre façon de nous intégrer avec le technologie. Des tâches langues naturelles à la vision par ordinateur, l'IA révolutionne la façon dont nous vivons et travaillons.

--- German Translation ---
Die Artikulierung der Informatik wird in die Wirklichkeit verwandelt. Von natürlichen Sprachverarbeitung bis hin zu Computer Vision, KI revolutioniert die Art und Weise, wie wir leben und arbeiten.

```

8. Implement a simple attention-based encoder-decoder model for English-to-Spanish translation using Tensorflow or PyTorch.

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torch.nn.functional as F
5
6 # 1. Toy Dataset (English -> Spanish)
7 data_pairs = [
8     ("the sky is blue", "el cielo es azul"),
9     ("the sun is bright", "el sol es brillante"),
10    ("the grass is green", "la hierba es verde"),
11    ("the night is dark", "la noche es oscura")
12 ]
13
14 # Helper function to build vocabulary
15 def build_vocab(sentences):
16     vocab = {"<pad>": 0, "<sos>": 1, "<eos>": 2}
17     for sentence in sentences:
18         for word in sentence.split():
19             if word not in vocab:
20                 vocab[word] = len(vocab)
21     return vocab
22
23 en_sentences, es_sentences = zip(*data_pairs)
24 en_vocab = build_vocab(en_sentences)
25 es_vocab = build_vocab(es_sentences)
26
27 en_idx2word = {i: w for w, i in en_vocab.items()}
28 es_idx2word = {i: w for w, i in es_vocab.items()}
29
30 # Vectorize and pad sequences
31 def preprocess(sentences, vocab, max_len=6):
32     padded_data = []
33     for s in sentences:
34         tokens = [vocab["<sos>"]] + [vocab[w] for w in s.split()] + [vocab["<eos>"]]
35         tokens += [vocab["<pad>"]] * (max_len - len(tokens))
36         padded_data.append(tokens[:max_len])
37     return torch.tensor(padded_data)
38
39 input_tensor = preprocess(en_sentences, en_vocab)
40 target_tensor = preprocess(es_sentences, es_vocab)

```

```

1 class Encoder(nn.Module):
2     def __init__(self, input_dim, emb_dim, hidden_dim):
3         super().__init__()
4         self.embedding = nn.Embedding(input_dim, emb_dim, padding_idx=0)
5         self.gru = nn.GRU(emb_dim, hidden_dim, batch_first=True)
6
7     def forward(self, src):

```

```

8         embedded = self.embedding(src)
9         outputs, hidden = self.gru(embedded)
10        return outputs, hidden # outputs: (batch, seq_len, hidden_dim)
11
12 class BahdanauAttention(nn.Module):
13     def __init__(self, hidden_dim):
14         super().__init__()
15         self.W1 = nn.Linear(hidden_dim, hidden_dim)
16         self.W2 = nn.Linear(hidden_dim, hidden_dim)
17         self.V = nn.Linear(hidden_dim, 1)
18
19     def forward(self, query, values):
20         query = query.transpose(0, 1) # (batch, 1, hidden_dim)
21
22         # Calculate attention scores
23         score = self.V(torch.tanh(self.W1(query) + self.W2(values))) # (batch, seq_len, 1)
24         attention_weights = F.softmax(score, dim=1) # (batch, seq_len, 1)
25
26         # Compute context vector
27         context_vector = attention_weights * values # (batch, seq_len, hidden_dim)
28         context_vector = torch.sum(context_vector, dim=1, keepdim=True) # (batch, 1, hidden_dim)
29
30         return context_vector, attention_weights
31
32 class AttnDecoder(nn.Module):
33     def __init__(self, output_dim, emb_dim, hidden_dim):
34         super().__init__()
35         self.attention = BahdanauAttention(hidden_dim)
36         self.embedding = nn.Embedding(output_dim, emb_dim, padding_idx=0)
37         self.gru = nn.GRU(emb_dim + hidden_dim, hidden_dim, batch_first=True)
38         self.fc_out = nn.Linear(hidden_dim, output_dim)
39
40     def forward(self, input_token, hidden, encoder_outputs):
41         # input_token: (batch, 1)
42         embedded = self.embedding(input_token) # (batch, 1, emb_dim)
43
44         # Calculate context vector using attention
45         context_vector, weights = self.attention(hidden, encoder_outputs)
46
47         # Combine embedded input token and context vector
48         rnn_input = torch.cat((embedded, context_vector), dim=2) # (batch, 1, emb_dim + hidden_dim)
49
50         output, hidden = self.gru(rnn_input, hidden)
51         prediction = self.fc_out(output.squeeze(1)) # (batch, output_dim)
52
53         return prediction, hidden, weights

```

```

1 # Model Hyperparameters
2 EMB_DIM = 16
3 HIDDEN_DIM = 32
4
5 encoder = Encoder(len(en_vocab), EMB_DIM, HIDDEN_DIM)
6 decoder = AttnDecoder(len(es_vocab), EMB_DIM, HIDDEN_DIM)
7
8 optimizer = optim.Adam(list(encoder.parameters()) + list(decoder.parameters()), lr=0.01)
9 criterion = nn.CrossEntropyLoss(ignore_index=0)
10
11 # Training execution
12 print("Training the Attention Model...")
13 for epoch in range(1, 151):
14     optimizer.zero_grad()
15     batch_size = input_tensor.size(0)
16     max_target_len = target_tensor.size(1)
17
18     encoder_outputs, encoder_hidden = encoder(input_tensor)
19     decoder_hidden = encoder_hidden
20
21     # Start sequence with <sos> token
22     decoder_input = target_tensor[:, 0].unsqueeze(1)
23     loss = 0
24
25     for t in range(1, max_target_len):
26         prediction, decoder_hidden, _ = decoder(decoder_input, decoder_hidden, encoder_outputs)
27         loss += criterion(prediction, target_tensor[:, t])
28         # Teacher Forcing: Feed the actual target token as the next input
29         decoder_input = target_tensor[:, t].unsqueeze(1)
30
31     loss.backward()

```



```

32     optimizer.step()
33
34     if epoch % 50 == 0:
35         print(f"Epoch {epoch}/150 | Total Sequence Loss: {loss.item():.4f}")
36
37 # Evaluation / Translation Step
38 print("\n--- Translating Verification Prompt ---")
39 encoder.eval()
40 decoder.eval()
41
42 with torch.no_grad():
43     # Evaluate on the first sample: "the sky is blue"
44     test_input = input_tensor[0:1]
45     enc_outputs, enc_hidden = encoder(test_input)
46     dec_hidden = enc_hidden
47
48     dec_input = torch.tensor([[es_vocab["<sos>"]]])
49     translated_words = []
50
51     for _ in range(6):
52         pred, dec_hidden, _ = decoder(dec_input, dec_hidden, enc_outputs)
53         next_token = pred.argmax(dim=1).item()
54
55         if next_token == es_vocab["<eos>"] or next_token == es_vocab["<pad>"]:
56             break
57         translated_words.append(es_idx2word[next_token])
58         dec_input = torch.tensor([[next_token]])
59
60     print(f"English Input:      {data_pairs[0][0]}")
61     print(f"Target Spanish:     {data_pairs[0][1]}")
62     print(f"Model Translation: { ' '.join(translated_words)}")

```

Training the Attention Model...

Epoch 50/150 | Total Sequence Loss: 0.0448

Epoch 100/150 | Total Sequence Loss: 0.0148

Epoch 150/150 | Total Sequence Loss: 0.0091

--- Translating Verification Prompt ---

English Input: the sky is blue

Target Spanish: el cielo es azul

Model Translation: el cielo es azul

9. Use the following short poetry dataset to simulate poem generation with a pre-trained GPT model:

["Roses are red, violets are blue,", "Sugar is sweet, and so are you.", "The moon glows bright in silent skies,", "A bird sings where the soft wind sighs."]

Using this dataset as a reference for poetic structure and language, generate a new 2-4 line poem using a pre-trained GPT model (such as GPT-2). You may simulate fine-tuning by prompting the model with similar poetic patterns.

```

1  from transformers import pipeline, set_seed
2
3  # Initialize the text-generation pipeline with GPT-2
4  generator = pipeline('text-generation', model='gpt2')
5
6  # Changed the seed to get a more poetic generation
7  set_seed(42)
8
9  prompt = (
10     "Poem 1:\n"
11     "Roses are red, violets are blue,\n"
12     "Sugar is sweet, and so are you.\n\n"
13     "Poem 2:\n"
14     "The moon glows bright in silent skies,\n"
15     "A bird sings where the soft wind sighs.\n\n"
16     "Poem 3:\n"
17 )
18
19 print("Generating new poem...\n")
20
21 output = generator(
22     prompt,
23     max_new_tokens=50,      # Increased to ensure it finishes both lines
24     temperature=0.7,        # Slightly lower temperature for better coherence
25     num_return_sequences=1,
26     pad_token_id=50256
27 )
28
29 full_text = output[0]['generated_text']
30 new_poem_raw = full_text.split("Poem 3:\n")[-1]
31

```



```

31
32 # Clean up the output by grabbing the first two non-empty lines
33 lines = [line.strip() for line in new_poem_raw.split('\n') if line.strip()]
34 new_poem = "\n".join(lines[:2])
35
36 print("--- Provided Dataset (Context) ---")
37 print("Roses are red, violets are blue,")
38 print("Sugar is sweet, and so are you.")
39 print("The moon glows bright in silent skies,")
40 print("A bird sings where the soft wind sighs.")
41 print("\n--- Generated Poem ---")
42 print(new_poem)

```

Loading weights: 100%

148/148 [00:00<00:00, 4200.12it/s]

[transformers] Both `max\_new\_tokens` (=50) and `max\_length` (=50) seem to have been set. `max\_new\_tokens` will take precedence. Generating new poem...

--- Provided Dataset (Context) ---
 Roses are red, violets are blue,
 Sugar is sweet, and so are you.
 The moon glows bright in silent skies,
 A bird sings where the soft wind sighs.

--- Generated Poem ---
 A man lies in the woods,
 He walks in a grassy plain

10. Imagine you are building a creative writing assistant for a publishing company. The assistant should generate story plots and character descriptions using Generative AI. Describe how you would design the system, including model selection, training data, bias mitigation, and evaluation methods. Explain the real-world challenges you might face.

1. Model Selection and Architecture To protect the publishing company's proprietary manuscripts and intellectual property, relying solely on public APIs (like OpenAI) poses data privacy risks.

Base Model: An open-weights Large Language Model (LLM) with a strong foundation in instruction-following and long-context processing, such as Llama 3 (8B or 70B) or Mistral.

Deployment: The model should be containerized using Docker and deployed on scalable cloud infrastructure like AWS EC2. This ensures the publisher retains complete ownership of the data pipeline and model weights.

Architecture: Implement a Retrieval-Augmented Generation (RAG) framework. When a user requests a character or plot, the system retrieves relevant tropes, pacing structures, or successful historical examples from a vector database (e.g., MongoDB with vector search) before generation.

2. Training and Reference Data A base model is too generic for professional publishing. It requires specialized data conditioning.

Pre-training / Base Data: The model inherently possesses knowledge of grammar, syntax, and public-domain literature (e.g., Shakespeare, Dickens).

Fine-Tuning Dataset: The company's historical backlist of successful novels, categorized by genre, pacing, and character archetypes.

Metadata Tags: Structured data outlining character motivations (e.g., "The Reluctant Hero"), story beats (e.g., "Save the Cat" beats), and world-building constraints.

3. Bias Mitigation Unchecked generative models often default to stereotypes, which can damage a publisher's reputation.

Curated Training Data: Actively audit the fine-tuning dataset to ensure representation across gender, ethnicity, and cultural backgrounds, removing historically prejudiced texts.

System Prompting: Inject hard constraints into the generation prompt (e.g., "Generate a character description that avoids cultural stereotypes and focuses on psychological depth").

Toxicity Classifiers: Use a secondary, lightweight NLP classifier layer to scan outputs for offensive, biased, or exclusionary language before presenting it to the writer.

4. Evaluation Methods Evaluating creative text is subjective, making traditional metrics like BLEU or ROUGE nearly useless.

Human-in-the-Loop (HITL): The primary metric is acceptance rate—how often editors and authors actually use, accept, or lightly modify the generated plots and characters versus regenerating them.

Diversity Metrics: Algorithmic checks to measure the distribution of demographic markers in generated characters over time to ensure the model isn't defaulting to homogenous traits.

Coherence and Consistency Checking: Using an automated evaluator (a separate LLM instance) to cross-check if the generated plot contains logical plot holes or if the character description contradicts itself.

5. Real-World Challenges Copyright and Plagiarism: The most severe risk is the model inadvertently memorizing and regurgitating copyrighted material from its training data. If an author publishes a generated plot that closely mirrors an existing intellectual